

```
"""
```

```
Enerji / Altyapı Portföy Dashboard
```

```
Python 3.10+ / Streamlit
```

```
Çalıştırma:
```

```
    pip install -r requirements.txt
```

```
    streamlit run app.py
```

```
"""
```

```
import datetime as dt
```

```
import numpy as np
```

```
import pandas as pd
```

```
import plotly.graph_objects as go
```

```
import requests
```

```
import streamlit as st
```

```
# -----
```

```
# SAYFA AYARLARI
```

```
# -----
```

```
st.set_page_config(
```

```
    page_title="Enerji / Altyapı Portföy Dashboard",
```

```
    layout="wide",
```

```
)
```

```
# -----
```

```
# BASİT ŞİFRE KORUMASI
```

```
# -----
```

```
def check_password() -> bool:
```

```
    """Kullanıcı doğru şifreyi girene kadar devam etmesini engeller."""
```

```
def password_entered():
```

```
    if st.session_state["password"] == st.secrets.get("dashboard_password", "
```

```
        st.session_state["password_correct"] = True
```

```
        del st.session_state["password"]
```

```
    else:
```

```
        st.session_state["password_correct"] = False
```

```
if st.session_state.get("password_correct", False):
```

```
    return True
```

```
st.text_input("Şifre", type="password", on_change=password_entered, key="pass
```

```

    if "password_correct" in st.session_state and not st.session_state["password_
        st.error("Şifre hatalı.")
    return False

if not check_password():
    st.stop()

ASSETS = ["ICLN", "RENW.L", "XLE", "UNG", "LNG"]
DEFAULT_WEIGHTS = {
    "ICLN": 0.30,
    "RENW.L": 0.20,
    "XLE": 0.25,
    "UNG": 0.15,
    "LNG": 0.10,
}

FRED_SERIES = {
    "ABD 10Y Faiz": "DGS10",
    "WTI Petrol": "DCOILWTICO",
    "Brent Petrol": "DCOILBRENTU",
    "Henry Hub Doğal Gaz": "DHHNGSP",
}

# Stooq sembolü; dolar endeksi için farklı kaynaklarda değişebilir, gerekirse gün
STOOQ_SYMBOLS = {
    "Dolar Endeksi (DXY proxy)": "usdx",
}

MIDAS_GUIDE = [
    {
        "Ticker": "FRO",
        "Temel Gösterge": "VLCC spot navlun / BDTI",
        "Anahtar Kelimeler": "tanker navlun, VLCC rate, spot freight, BDTI",
        "Nasıl Okunur": "Navlun oranları yükseliyorsa taşıma geliri artar.",
        "Tipik İlişki": "Pozitif: navlun ↑ → FRO geliri ↑",
    },
    {
        "Ticker": "TRMD",
        "Temel Gösterge": "Product tanker TCE",
        "Anahtar Kelimeler": "product tanker, TCE, navlun endeksi",
        "Nasıl Okunur": "TCE (time-charter equivalent) yükseldikçe karlılık artar",
        "Tipik İlişki": "Pozitif: TCE ↑ → TRMD marjı ↑",
    },
    {
        "Ticker": "EPD",
        "Temel Gösterge": "Throughput / terminal doluluk",
    }
]

```

```

        "Anahtar Kelimeler": "throughput, terminal utilization, doluluk oranı, mi
        "Nasıl Okunur": "Yüksek doluluk oranı istikrarlı nakit akışına işaret ede
        "Tipik İlişki": "Pozitif: doluluk ↑ → nakit akışı istikrarı ↑",
    },
    {
        "Ticker": "TUPRS",
        "Temel Gösterge": "Crack spread / rafineri marjı",
        "Anahtar Kelimeler": "crack spread, rafineri marjı, refining margin",
        "Nasıl Okunur": "Crack spread genişledikçe rafineri karlılığı artar.",
        "Tipik İlişki": "Pozitif: crack spread ↑ → TUPRS marjı ↑",
    },
    {
        "Ticker": "OIH",
        "Temel Gösterge": "Rig count / E&P CAPEX",
        "Anahtar Kelimeler": "rig count, drilling activity, E&P capex",
        "Nasıl Okunur": "Artan rig sayısı, servis şirketlerine talep artışı demek
        "Tipik İlişki": "Pozitif: rig count ↑ → OIH gelirleri ↑",
    },
]

```

```

STRATEGIC_GUIDE = [
    {
        "Ticker": "ICLN",
        "Temel Gösterge": "Temiz enerji endeksi performansı",
        "İzleme Mantığı": "Küresel temiz enerji hisse sepetini takip eder.",
        "Tipik İlişki": "Faiz oranlarıyla negatif korelasyon eğilimi.",
    },
    {
        "Ticker": "RENW.L",
        "Temel Gösterge": "Avrupa temiz enerji performansı",
        "İzleme Mantığı": "Londra listeli, Avrupa ağırlıklı temiz enerji sepeti."
        "Tipik İlişki": "Avrupa enerji politikalarıyla pozitif ilişkili.",
    },
    {
        "Ticker": "XLE",
        "Temel Gösterge": "ABD enerji sektörü (S&P Energy)",
        "İzleme Mantığı": "Büyük ABD enerji şirketlerini kapsar.",
        "Tipik İlişki": "WTI/Brent fiyatlarıyla pozitif korelasyon.",
    },
    {
        "Ticker": "UNG",
        "Temel Gösterge": "Doğal gaz fiyat takibi",
        "İzleme Mantığı": "Henry Hub vadeli işlemlerini takip eder.",
        "Tipik İlişki": "Henry Hub fiyatlarıyla doğrudan pozitif ilişki.",
    },
    {
        "Ticker": "LNG",

```

```

        "Temel Gösterge": "Cheniere Energy operasyonel performansı",
        "İzleme Mantığı": "LNG ihracat kapasitesi ve sözleşme gelirleri.",
        "Tipik İlişki": "Küresel LNG spot fiyatlarıyla pozitif ilişki.",
    },
]

# -----
# VERİ ÇEKME FONKSİYONLARI
# -----

@st.cache_data(ttl=8 * 3600, show_spinner=False)
def fetch_yahoo_prices(ticker: str, start_date: dt.date) -> pd.DataFrame:
    """Yahoo Finance chart endpoint üzerinden günlük adj-close verisi çeker."""
    period1 = int(dt.datetime.combine(start_date, dt.time.min).timestamp())
    period2 = int(dt.datetime.now().timestamp())
    url = f"https://query1.finance.yahoo.com/v8/finance/chart/{ticker}"
    params = {
        "period1": period1,
        "period2": period2,
        "interval": "1d",
        "events": "div,splits",
    }
    headers = {"User-Agent": "Mozilla/5.0"}

    resp = requests.get(url, params=params, headers=headers, timeout=15)
    resp.raise_for_status()
    data = resp.json()

    result = data.get("chart", {}).get("result")
    if not result:
        raise ValueError(f"{ticker} için veri bulunamadı.")

    result = result[0]
    timestamps = result.get("timestamp")
    if not timestamps:
        raise ValueError(f"{ticker} için zaman serisi boş.")

    indicators = result["indicators"]
    adjclose = indicators.get("adjclose", [{}])[0].get("adjclose")
    close = indicators["quote"][0].get("close")
    prices = adjclose if adjclose else close

    df = pd.DataFrame({"date": pd.to_datetime(timestamps, unit="s").date, "close"
    df = df.dropna(subset=["close"])
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date").drop_duplicates(subset="date").set_index("date")

```

```

return df

@st.cache_data(ttl=8 * 3600, show_spinner=False)
def fetch_fred_series(series_id: str, start_date: dt.date) -> pd.DataFrame:
    """FRED'in API-key gerektirmeyen CSV export endpoint'inden seri çeker."""
    url = f"https://fred.stlouisfed.org/graph/fredgraph.csv?id={series_id}"
    resp = requests.get(url, timeout=15)
    resp.raise_for_status()

    from io import StringIO

    df = pd.read_csv(StringIO(resp.text))
    df.columns = ["date", "value"]
    df["date"] = pd.to_datetime(df["date"])
    df["value"] = pd.to_numeric(df["value"], errors="coerce")
    df = df.dropna(subset=["value"])
    df = df[df["date"] >= pd.to_datetime(start_date)]
    return df.set_index("date")

@st.cache_data(ttl=8 * 3600, show_spinner=False)
def fetch_stooq_series(symbol: str, start_date: dt.date) -> pd.DataFrame:
    """Stooq CSV download endpoint'inden günlük seri çeker."""
    url = f"https://stooq.com/q/d/l/?s={symbol}&i=d"
    resp = requests.get(url, timeout=15)
    resp.raise_for_status()

    from io import StringIO

    df = pd.read_csv(StringIO(resp.text))
    df.columns = [c.lower() for c in df.columns]
    df["date"] = pd.to_datetime(df["date"])
    df = df[df["date"] >= pd.to_datetime(start_date)]
    return df.set_index("date")[["close"]]

# -----
# HESAPLAMA FONKSİYONLARI
# -----

def compute_returns(price_df: pd.DataFrame) -> pd.DataFrame:
    return price_df.pct_change().dropna()

def compute_portfolio_index(returns: pd.DataFrame, weights: dict) -> pd.Series:

```

```

w = pd.Series(weights)
w = w / w.sum()
port_returns = (returns[w.index] * w).sum(axis=1)
port_index = (1 + port_returns).cumprod() * 100
return port_index

def base_100(series: pd.Series) -> pd.Series:
    return series / series.iloc[0] * 100

def annualized_volatility(returns: pd.Series) -> float:
    return returns.std() * np.sqrt(252)

def max_drawdown(index_series: pd.Series) -> float:
    peak = index_series.cummax()
    dd = index_series / peak - 1
    return dd.min()

def total_return(index_series: pd.Series) -> float:
    return index_series.iloc[-1] / index_series.iloc[0] - 1

# -----
# SIDEBAR
# -----

st.sidebar.header("Ayarlar")

portfolio_value = st.sidebar.number_input(
    "Toplam portföy tutarı (USD)", min_value=0.0, value=1_000_000.0, step=10_000.
)
energy_weight_pct = st.sidebar.selectbox("Enerji/altyapı ağırlığı", [15, 20, 25],
start_date = st.sidebar.date_input(
    "Başlangıç tarihi", value=dt.date.today() - dt.timedelta(days=5 * 365)
)

st.sidebar.caption("Sepet içi varlık ağırlıkları sabit: ICLN %30, RENW.L %20, XLE

st.title("Enerji / Altyapı Portföy Dashboard")

# -----
# FİYAT VERİSİ ÇEKME
# -----

```

```

price_data = {}
failed_assets = []

with st.spinner("Fiyat verileri çekiliyor..."):
    for ticker in ASSETS:
        try:
            df = fetch_yahoo_prices(ticker, start_date)
            price_data[ticker] = df["close"]
        except Exception:
            failed_assets.append(ticker)

if len(price_data) < 3:
    st.error(
        "Veri alınamadı, API/bağlantı kontrol edilmelidir. "
        f"Başarısız olan varlıklar: {'', ' '.join(failed_assets) if failed_assets el
    )
    st.stop()

if failed_assets:
    st.warning(f"Şu varlıklar için veri alınamadı ve hesaplamalara dahil edilmedi

price_df = pd.DataFrame(price_data).dropna(how="all").ffill().dropna()
available_assets = list(price_df.columns)
weights = {k: v for k, v in DEFAULT_WEIGHTS.items() if k in available_assets}

returns = compute_returns(price_df)
portfolio_index = compute_portfolio_index(returns, weights)

# -----
# KPI KUTULARI
# -----

energy_basket_amount = portfolio_value * (energy_weight_pct / 100)
port_total_return = total_return(portfolio_index)
port_vol = annualized_volatility(portfolio_index.pct_change().dropna())
port_max_dd = max_drawdown(portfolio_index)

col1, col2, col3, col4 = st.columns(4)
col1.metric("Enerji Sepeti Tutarı", f"${energy_basket_amount:,.0f}")
col2.metric("5Y Toplam Getiri", f"{port_total_return * 100:.1f}%")
col3.metric("Yıllık Volatilité", f"{port_vol * 100:.1f}%")
col4.metric("Maks. Düşüş", f"{port_max_dd * 100:.1f}%")

# -----
# ANA GRAFİK: BAZ 100 PERFORMANS
# -----

```

```

st.subheader("Baz 100 Performans Karşılaştırması")

fig = go.Figure()
fig.add_trace(
    go.Scatter(x=portfolio_index.index, y=portfolio_index.values, name="Portföy",
)
for ticker in available_assets:
    series_100 = base_100(price_df[ticker])
    fig.add_trace(go.Scatter(x=series_100.index, y=series_100.values, name=ticker

fig.update_layout(height=500, margin=dict(l=10, r=10, t=30, b=10), hovermode="x u
st.plotly_chart(fig, use_container_width=True)

# -----
# KORELASYON + ÖZET TABLO
# -----

col_left, col_right = st.columns(2)

with col_left:
    st.subheader("Korelasyon Matrisi")
    corr = returns.corr()
    heatmap = go.Figure(
        data=go.Heatmap(
            z=corr.values,
            x=corr.columns,
            y=corr.columns,
            zmin=-1,
            zmax=1,
            colorscale="RdBu",
            text=np.round(corr.values, 2),
            texttemplate="%{text}",
        )
    )
    heatmap.update_layout(height=400, margin=dict(l=10, r=10, t=10, b=10))
    st.plotly_chart(heatmap, use_container_width=True)

with col_right:
    st.subheader("Varlık Özeti")
    summary_rows = []
    for ticker in available_assets:
        series_100 = base_100(price_df[ticker])
        summary_rows.append(
            {
                "Varlık": ticker,
                "Ağırlık": f"{weights.get(ticker, 0) * 100:.0f}%",
                "Son Fiyat": f"{price_df[ticker].iloc[-1]:.2f}",
            }
        )

```



```

        "5Y Getiri": f"{total_return(series_100) * 100:.1f}%",
        "Volatilite": f"{annualized_volatility(returns[ticker]) * 100:.1f}%",
        "Maks. Düşüş": f"{max_drawdown(series_100) * 100:.1f}%",
    }
)
st.dataframe(pd.DataFrame(summary_rows), use_container_width=True, hide_index

# -----
# GETİRİ VS VOLATİLİTE (RİSK) SCATTER
# -----

st.subheader("Getiri vs Volatilite")
st.caption("Her varlığın 5Y toplam getirisi ile yıllık volatilitesi (risk) karşıl

scatter_points = []
for ticker in available_assets:
    series_100 = base_100(price_df[ticker])
    scatter_points.append(
        {
            "Varlık": ticker,
            "Volatilite": annualized_volatility(returns[ticker]) * 100,
            "Getiri": total_return(series_100) * 100,
        }
    )
scatter_points.append(
    {
        "Varlık": "Portföy",
        "Volatilite": port_vol * 100,
        "Getiri": port_total_return * 100,
    }
)

scatter_df = pd.DataFrame(scatter_points)

fig_scatter = go.Figure()
for _, row in scatter_df.iterrows():
    is_portfolio = row["Varlık"] == "Portföy"
    fig_scatter.add_trace(
        go.Scatter(
            x=[row["Volatilite"]],
            y=[row["Getiri"]],
            mode="markers+text",
            text=[row["Varlık"]],
            textposition="top center",
            marker=dict(
                size=16 if is_portfolio else 12,
                symbol="diamond" if is_portfolio else "circle",
            )
        )
    )

```

```

        color="#FFD700" if is_portfolio else None,
    ),
    name=row["Varlık"],
    showlegend=False,
)
)

fig_scatter.update_layout(
    height=450,
    margin=dict(l=10, r=10, t=10, b=10),
    xaxis_title="Yıllık Volatilité (%)",
    yaxis_title="5Y Toplam Getiri (%)",
)
st.plotly_chart(fig_scatter, use_container_width=True)

# -----
# SEKMELER
# -----

tab1, tab2, tab3, tab4 = st.tabs(
    ["Makro Göstergeler", "Gösterge Rehberi", "Midas Takip Ekranı", "Ham Veri / E
")

with tab1:
    st.caption("FRED ve Stooq üzerinden çekilen makro seriler.")
    for label, series_id in FRED_SERIES.items():
        try:
            fred_df = fetch_fred_series(series_id, start_date)
            fig_macro = go.Figure(go.Scatter(x=fred_df.index, y=fred_df["value"],
            fig_macro.update_layout(title=label, height=250, margin=dict(l=10, r=
            st.plotly_chart(fig_macro, use_container_width=True)
        except Exception:
            st.warning(f"{label} verisi şu anda çekilemedi (FRED bağlantı sorunu)

    for label, symbol in STOOQ_SYMBOLS.items():
        try:
            stooq_df = fetch_stooq_series(symbol, start_date)
            fig_macro = go.Figure(go.Scatter(x=stooq_df.index, y=stooq_df["close"
            fig_macro.update_layout(title=label, height=250, margin=dict(l=10, r=
            st.plotly_chart(fig_macro, use_container_width=True)
        except Exception:
            st.warning(f"{label} verisi şu anda çekilemedi (Stooq bağlantı sorunu

with tab2:
    st.caption("Stratejik portföydeki varlıklar için temel izleme göstergeleri.")
    st.dataframe(pd.DataFrame(STRATEGIC_GUIDE), use_container_width=True, hide_in

```

```

with tab3:
    st.caption("Midas'ta arama yaparken kullanılacak anahtar kelime rehberi.")
    st.dataframe(pd.DataFrame(MIDAS_GUIDE), use_container_width=True, hide_index=
    st.markdown(
        """
        **Kısa Özet:**
        - FRO / TRMD → navlun / TCE sinyali
        - EPD → hacim / doluluk
        - TUPRS → crack spread
        - OIH → CAPEX / rig count
        """
    )

with tab4:
    st.caption("Fiyat serileri ve portföy endeksi.")
    export_df = price_df.copy()
    export_df["Portföy Endeksi"] = portfolio_index
    st.dataframe(export_df.tail(30), use_container_width=True)

    csv_data = export_df.to_csv().encode("utf-8")
    st.download_button(
        "CSV olarak indir",
        data=csv_data,
        file_name="portfoy_verisi.csv",
        mime="text/csv",
    )

```